

From Models to Frontiers — Volume III

THE LLM TRILOGY

From Models to Frontiers

Advanced topics in large language modeling

VOLUME III · ADVANCED

Sharif Uddin

From Models to Frontiers

Sharif Uddin

Volume III of the LLM trilogy · Volume I · Volume II

Most people who build with language models never need to look inside. They pick an API, craft a system prompt, add a retrieval layer, and ship. Volume II covered exactly that ground — and it covers most of what most practitioners need.

This volume is for the readers who hit the wall. The ones who need to evaluate whether a published scaling result applies to their domain. Who want to argue about alignment tradeoffs rather than just implement a safety layer. Who need to understand why their inference costs scaled badly, or why their fine-tuned model broke in a specific way. Who read a paper abstract and want to know whether to trust the benchmark.

From Models to Frontiers steps behind the API surface and into the science and engineering of the models themselves — why they are built the way they are, what the research says about their limits, and what is genuinely unknown.

Who this book is for

You build or evaluate systems, and you need depth. You can follow a discussion of fine-tuning, RLHF, and retrieval without needing every term re-explained. You have read model cards and benchmark reports and developed a healthy suspicion of both. You want to be the person in the room who can engage with the paper, not just nod at the headline.

You work in AI safety, alignment, or policy. You need the technical vocabulary to participate in technical debates — what alignment actually means in a training pipeline, what capability evaluations can and cannot tell you, where the science is genuinely uncertain versus where it is settled.

You make infrastructure or research decisions. You need to evaluate efficiency claims, understand the tradeoffs between training and inference strategies, and decide when vendor roadmaps matter to your situation and when they are irrelevant.

You are a senior contributor who reads the field. You want a structured map of the frontier — enough to follow primary sources with judgment, to recognize hype, to decide where your own learning should go next, and to bring research context into product and engineering conversations.

You do not need this book if: - You are building LLM features with APIs and the Vol II coverage of prompting, RAG, evaluation, and deployment is sufficient for your work. - You want a textbook treatment of neural networks from first principles. This book connects research to practice, not mathematics to code. - You are comfortable with the research literature and just need a quick reference. Start with the part that covers your gap.

What this book is for

The boundary between Volume II and Volume III is the boundary between *building with models* and *understanding or critically evaluating the models themselves*.

That boundary matters in specific ways:

Evaluating claims. When a new model is released with a 40% improvement on a reasoning benchmark, you want to know whether that benchmark is contaminated, whether the evaluation methodology favors their training distribution, and whether the improvement translates to your use case. That judgment requires understanding how models are trained, evaluated, and where benchmarks go wrong.

Making architectural decisions. When you need to choose between RAG and fine-tuning, or between serving a large model and distilling it into a smaller one, the decision depends on understanding what fine-tuning actually does to a model's knowledge and behavior — not just the API surface.

Understanding failure modes. Sycophancy, hallucination, prompt injection vulnerability, alignment failures under distribution shift — these are product problems, but they have technical roots. Understanding the roots lets you design mitigations rather than patches.

Participating in research and governance. If your work involves alignment decisions, capability evaluations, release policy, or safety specifications, you need the vocabulary that comes from the research side of these conversations.

How this volume is organized

The five parts form a progression from *how models are built* to *what they are becoming* to *how to engage with the field*.

Part I — Scale, data, and the pretraining stack: Why do large models look the way they do? Scaling laws, training data at web scale, architecture families, and the post-training pipeline that transforms a base model into something deployable.

Part II — Alignment, safety, and robustness: What does it mean to align a model, and what can go wrong? Goals and tensions in alignment, adversarial evaluation, interpretability tools, and the governance landscape around model release and dual-use.

Part III — Efficiency: Training systems, inference optimization, and hardware context. What techniques make large models affordable to serve, when do they trade quality, and how do infrastructure decisions filter which methods are practical at your scale.

Part IV — Beyond text: Multimodal models, tool-augmented agents, and the research challenges they introduce. Vision-language architectures, grounding at research depth, and the failure modes of multi-step agent systems.

Part V — Frontiers and open problems: Where does the field not know the answers? Evaluation at the frontier, open research directions, and how to build a sustainable reading practice without drowning in preprints.

Prerequisites

This volume assumes you are comfortable with: - The vocabulary and practices in Volume II: prompting, RAG, fine-tuning decisions, evaluation, deployment - High-level discussions of training: that models are pretrained on text, that instruction tuning and preference optimization exist, and that they have different effects - Reading research abstracts critically: understanding what a benchmark evaluates, what “RLHF” means in a paper headline, what “we fine-tuned on X” implies

You do not need to derive backpropagation or read CUDA kernels. Where topics depend on linear algebra, distributed systems, or statistics at depth, the text signals it and points to standard references rather than rederiving everything from scratch.

Full text — Parts I through V

Part I — Scale, data, and the pretraining stack

Sharif Uddin

From Models to Frontiers · Volume III

To understand why large models behave the way they do — why they are good at some things and brittle on others, why updates sometimes break behavior that was working, why two models with similar parameter counts can perform very differently — you need to understand how they are built. This part covers the scientific and engineering forces that shape modern foundation models: scaling trends, the data pipelines that feed them, the architecture decisions that have survived (and those that are being revisited), and the post-training stages that transform a raw base model into something a team can ship.

Contents of this part

In the full volume table of contents, these correspond to sections 1–4.

	Chapter	What you will take away
1	Scaling laws and predictable returns	How compute, data, and parameters interact; Chinchilla intuition; limits and domain shifts
2	Data at scale: curation, contamination, documentation	Web crawls, filtering, deduplication, leakage, datasheets
3	Architecture variants and inductive biases	Transformers, sparse MoE, long-context methods, state-space hybrids
4	Post-training: beyond next-token prediction	Instruction tuning, preference optimization, multistage pipelines

Contents (plain list — same as table):

- 1. Scaling laws — compute-optimal allocation and the limits of power laws.
 - 2. Data at scale — curation, deduplication, contamination, documentation.
 - 3. Architecture variants — transformers, MoE, long-context, state-space models.
 - 4. Post-training — SFT, RLHF/DPO, iterative pipelines.
-

Chapter 1 — Scaling laws and predictable returns

A scaling curve is not a roadmap — it is a measurement of what happened under specific conditions, not a promise about what will happen under yours.

When researchers discovered that language model performance follows smooth power-law relationships — that loss decreases predictably as you increase model size, training data, or compute — it was genuinely exciting. You could, with some care, extrapolate the training loss of a model you had not built yet. You could allocate compute budgets with something better than intuition. The field found a kind of empirical physics for model behavior.

The Chinchilla result

For most of the early scaling era, larger models were the goal: given a fixed compute budget, use it to train the biggest model you could afford, even if that meant training it on relatively little data. The Chinchilla research (Hoffmann et al., 2022) challenged this by asking: for a fixed compute budget, what is the optimal allocation between model size and training tokens?

The answer was roughly: balance them. Earlier large models were undertrained — too many parameters, not enough tokens. A smaller model trained on more data could match or exceed a larger undertrained model at the same compute cost, and at lower inference cost (since it is smaller).

The practical implications: - **Compute-optimal training** means running more epochs or collecting more data, not just building a bigger model. - **Inference economics** matter at planning time. A smaller, well-trained model costs less to serve for every query after training. - **The specific ratios vary** — the Chinchilla numbers are not universal constants. Recipe, architecture, data domain, and objectives all affect the optimal balance.

Use Chinchilla intuitions as *planning tools*, not commandments. The most important insight is the principle: do not assume that scale in one dimension (parameters) substitutes for scale in another (data).

Power laws and their limits

Scaling laws describe relationships within a training setup, held roughly constant: - Same architecture family - Same data distribution - Same training objective - Similar hyperparameter tuning

Change any of these and the curve shifts. This matters because:

Domain shift breaks naive extrapolation. A scaling law measured on general web text does not tell you how much data or compute you need for a code model, a biomedical model, or a multilingual model. The domain-specific distribution determines what the model learns from each token.

Data quality beats raw token count at domain boundaries. A model trained on 10 billion carefully curated legal documents may outperform one trained on 100 billion noisily filtered general-purpose tokens on legal tasks. The loss curves will not tell you this.

Benchmark saturation distorts scaling plots. When a benchmark approaches ceiling performance, the apparent scaling benefit disappears — not because scaling stopped working, but because the metric stopped resolving differences. Scaling results cited on saturated benchmarks are noise.

Inference and alignment costs may dominate product economics. Even when training scales efficiently, the cost per query at inference time and the engineering cost of aligning the model for deployment may dwarf training costs once a model is serving millions of users. Scaling training is only part of the cost picture.

Friction: “We trained a bigger model” is sometimes a response to what is actually a distribution shift problem, a data quality problem, or a post-training problem. Scaling cannot apologize for the wrong data.

What scaling laws are actually useful for

Scaling laws give you: - A principled way to allocate compute between model size and training tokens. - A way to predict approximate loss at scale before committing a large experiment. - A framework for understanding why some approaches work better at large scale than small scale (and vice versa).

They do not give you: - Guarantees about specific downstream task performance. - Transfer across data distributions. - Predictions about emergent capabilities that do not appear in the loss curve.

Takeaway

- Scaling laws are empirical tools for planning experiments and allocating compute — not universal constants.
 - Chinchilla-style intuition: for a fixed compute budget, balance model size and training tokens.
 - Domain shift, data quality, benchmark saturation, and inference economics all break naive scaling extrapolations.
 - Measure on your task and distribution; do not assume the scaling curve generalizes.
-

Chapter 2 — Data at scale: curation, contamination, and documentation

The quality of a foundation model is bounded by the quality of its training data, but “quality” is not a single axis.

Pretraining a large language model requires data at a scale that makes careful hand-curation impossible. Common English web crawls run to trillions of tokens. The practical question is not “curate everything manually” but “which automated and semi-automated filtering decisions produce a training distribution that leads to capable, reliable, and safe model behavior?”

The web crawl pipeline

Most large language models are pretrained on derivatives of web crawls (Common Crawl being the most common source), typically combined with curated corpora — books, scientific papers, code repositories, Wikipedia, and licensed datasets. The pipeline from raw crawl to training tokens involves:

Language filtering. Identifying and selecting (or weighting) languages. Most pipelines use fastText or similar classifiers. Choice of target languages and their balance shapes multilingual capability significantly.

Quality filtering. Removing low-quality content: spam, auto-generated SEO text, very short or very long documents, content with high repetition rates, non-natural-language content. Quality classifiers are often bootstrapped from curated high-quality sources — scoring documents by their similarity to good reference corpora.

Deduplication. Exact and near-duplicate removal. This matters for at least two reasons: exact duplicates inflate the effective weight of specific facts and phrasing, and contamination detection (covered below) requires knowing whether benchmark text appeared in training. Near-dedup typically uses MinHash or similar locality-sensitive hashing.

Toxicity and PII filtering. Removing or downweighting content that contains slurs, harassment, explicit content, or personally identifiable information. These decisions are imperfect — aggressive filtering removes relevant content, and no filter catches everything.

Each of these decisions involves tradeoffs. Aggressive deduplication can remove useful repetition (concepts that appear frequently benefit from that frequency). Aggressive quality filtering can skew rare languages and domains that produce less English-like writing styles. The pipeline encodes values, whether the team intends it to or not.

Contamination and benchmark integrity

Test-set contamination is the accidental (or deliberate) inclusion of benchmark evaluation examples in training data. When benchmark text appears in the crawl and the model was trained on that crawl, the model has effectively seen the “exam questions” during training. Reported performance on that benchmark is now a measure of memorization, not generalization.

Contamination is hard to detect definitively and easy to overlook. Common signals: - Performance on a new benchmark is suspiciously higher than closely related tasks. - The training cutoff predates the benchmark release but the crawl was assembled later. - Performance drops when evaluated on held-out

variations of the benchmark.

Responsible reporting includes a contamination analysis — checking whether benchmark text overlaps with training data — and should be cited when interpreting published results. Benchmarks released after a training cutoff are more trustworthy, but not immune to contamination from later model versions trained on more recent data.

Memorable detail: leaderboard jumps have occurred when benchmark text was in the crawl — not because the model “understood” more, but because the exam was accidentally open-book. The model was not cheating intentionally; the pipeline just failed to exclude it.

Data documentation: datasheets and data statements

Datasheets for datasets (Gebru et al., 2018) proposed that datasets should include structured documentation of their motivation, composition, collection process, preprocessing steps, intended uses, and limitations — analogous to product documentation for manufactured components. The concept has been widely endorsed and inconsistently adopted.

For foundation models, meaningful data documentation should include: - **Provenance:** Where did the data come from? What sources were included and excluded? - **Time range:** What is the training data cutoff? What temporal biases might this introduce? - **Language and domain distribution:** How are different languages, registers, and domains represented? - **Filtering decisions:** What content was removed, and by what criteria? - **Known limitations and biases:** What groups or topics are underrepresented or skewed?

This documentation is not just for academic completeness. It shapes what practitioners can trust the model to know, where its knowledge cuts off, which domains require supplementary retrieval or fine-tuning, and where claims about model behavior should be interrogated rather than accepted.

Anchor: A model’s capabilities and biases are downstream of its training data. Understanding the data is part of understanding the model.

Takeaway

- Web-scale data pipelines involve quality filtering, deduplication, toxicity filtering, and language selection — each a value-laden decision.
 - Contamination of benchmark test sets in training data distorts evaluation results. Check for it; report it.
 - Datasheets and data documentation are not bureaucracy — they are the infrastructure for honest evaluation and responsible deployment.
 - Data quality and distribution matter as much as raw token count for domain-specific capabilities.
-

Chapter 3 — Architecture variants and inductive biases

The transformer architecture has dominated language modeling for several years, but “transformer” covers a wide range of variants, and several competing families have developed strong arguments in specific niches.

Understanding architectures at survey depth — knowing what problem each family addresses and what it trades off — matters for evaluating model releases, understanding capability claims, and making decisions about which model families to build on.

The transformer baseline

The standard transformer for language modeling uses a decoder-only architecture with causal (autoregressive) attention: each token attends to all previous tokens in the sequence. Key components:

- **Self-attention:** Each token position computes a weighted sum over all previous positions. This is what enables long-range dependency modeling — a word at the end of a paragraph can directly attend to relevant context at the beginning.
- **Feed-forward layers:** Dense transformations applied at each position independently. These have been shown to store factual knowledge and perform position-wise computations.
- **Layer normalization and residual connections:** Stabilize training and enable the deep stacking of many layers.

The computational cost of attention is quadratic in sequence length — a 10x longer sequence requires 100x more attention computation. This is manageable for typical context lengths but becomes a constraint for very long contexts.

Mixture-of-experts (MoE)

Instead of activating all model parameters for every token, a MoE architecture routes each token to a subset of “expert” feed-forward layers. A small router network decides which experts are active. This allows a model to have a much larger total parameter count (and therefore more total capacity) while activating only a fraction of parameters per forward pass.

The appeal: you can scale total capacity without scaling per-token compute proportionally. The costs: routing overhead, expert load balancing (preventing all tokens from routing to the same few experts), and increased complexity in distributed training.

Direct address: When a model release says “2 trillion parameters, activates 220 billion per token” — that is a MoE model. Total parameters and active parameters are different numbers, and both matter for different reasons.

Long-context methods

Extending the effective context window beyond the length models were trained on requires attention to several problems:

Positional encoding generalization. Most transformers encode position with learned or fixed embeddings that may not generalize to positions beyond those seen in training. Techniques like RoPE (rotary position encoding) and ALiBi provide better length generalization, and are now standard in most large models.

Attention approximations. For very long contexts, full quadratic attention becomes prohibitively expensive. Approaches include sliding window attention (attending only to nearby tokens), sparse attention patterns (learned or fixed masks), and linear approximations to attention.

KV cache pressure. Long contexts require storing large key-value caches during inference (see Part III for detail). This creates memory constraints that affect deployment choices independent of training context length.

State-space models and hybrids

State-space models (SSMs), particularly the Mamba architecture and its descendants, offer a different approach to sequence modeling: instead of attention, they use recurrent state transitions that scale linearly in sequence length. For long sequences where full attention is expensive, SSMs can be more efficient.

The trade-offs: SSMs process the sequence recurrently and do not have full access to arbitrary earlier positions in the way attention does. This may affect performance on tasks that require attending to very specific positions from early in a long context. Hybrid architectures that combine attention layers with SSM layers attempt to capture both strengths.

Scale vs prior

An important question for any architecture decision: is a capability the result of scale, or is it built in by architecture and objective choice?

Some capabilities improve predictably with scale in a given architecture. Others are more dependent on architecture design — long-range coherence, for example, depends significantly on how attention is designed. Separating scale effects from architectural priors requires controlled comparisons that vary one dimension at a time, which is expensive and rarely done cleanly in practice.

Anchor: Architecture debates are interesting in research. In practice, the question is what measures move on your workload. Evaluate, do not just logos.

Takeaway

- Decoder-only transformers remain the dominant architecture for language modeling. Most frontier models are variants of this family.
- Mixture-of-experts enables larger total capacity at similar per-token compute; introduces routing complexity.
- Long-context methods (positional encoding, attention approximations) extend usable context but involve tradeoffs.
- State-space and hybrid architectures offer efficient long-context processing; trade-offs in retrieval from arbitrary positions.
- Architecture is a lever; empirical evaluation on your workload is the arbiter.

Chapter 4 — Post-training: beyond next-token prediction

Pretraining produces a model that is very good at predicting the next token in text that looks like the internet. That is not the same as a model that is helpful, honest, and safe to deploy.

The post-training pipeline is what transforms a base model into something aligned with user expectations and organizational requirements. It is also where many of the failure modes that practitioners encounter are introduced — and where mitigations can be applied.

Instruction tuning (supervised fine-tuning)

The first post-training stage for most deployable models is supervised fine-tuning (SFT) on demonstrations of desired behavior. A dataset of (instruction, response) pairs is used to fine-tune the base model so that it follows instructions, responds in dialogue format, and handles common task types.

SFT teaches: - **Task format:** How to interpret user instructions and structure responses. - **Dialogue conventions:** Turn-taking, conversation context, how to ask for clarification. - **Basic refusals:** How to decline obviously harmful requests.

SFT does not fundamentally change what the model knows — that came from pretraining. It changes how the model expresses and applies that knowledge in response to instruction. A model that learned about chemistry during pretraining will still know chemistry after SFT; SFT affects whether it presents that knowledge helpfully, in response to requests, in an appropriate format.

Important nuance: Over-tuning on SFT data can lead to “format lock” — the model applies the patterns from the demonstrations even when they do not fit, producing verbose, formulaic responses that hit the template but miss the point.

Preference optimization

After SFT, models still exhibit behaviors that are subtly wrong: sycophancy (agreeing with users rather than giving accurate information), over-refusal (declining reasonable requests), under-refusal (agreeing to requests they should decline), verbosity, and inconsistency.

RLHF (reinforcement learning from human feedback) addresses this by: 1. Collecting human preferences: presenting raters with pairs of model outputs and asking which is better. 2. Training a reward model on these preferences. 3. Using reinforcement learning to fine-tune the policy model to maximize the reward model’s score.

DPO (direct preference optimization) achieves a similar goal without a separate reward model training stage, by directly optimizing a policy that prefers the “chosen” response over the “rejected” response from the preference dataset.

Both approaches have important limitations: - **Goodhart’s Law:** The model learns to maximize what the reward model measures, which is a proxy for human preference. If the proxy is wrong (e.g., the reward model prefers verbose responses), the model learns to be verbose. - **Sycophancy as reward hacking:**

Human raters often prefer confident, agreeable responses. Models trained to maximize preference scores may learn to be agreeable rather than accurate. - **Distribution sensitivity:** Preference optimization on one distribution of prompts may not generalize well to different prompt distributions.

Tiny vignette: “One more epoch of SFT does not fix a preference-stage failure.” If a model is sycophantic, that is usually a preference optimization problem — it learned that agreeable responses score well. Adding more SFT data about being honest does not fix the reward signal that incentivizes agreement.

Multistage pipelines and iterative refinement

Modern post-training pipelines are not single-stage. A typical sequence:

```
Pretraining
↓
SFT (instruction format, basic alignment)
↓
Preference optimization (RLHF or DPO)
↓
Evaluation gate (quality + safety thresholds)
↓
Optional: iterative rounds with updated feedback data
↓
Deployment
```

Each stage can shift both capabilities and failure modes. Changes introduced by SFT can interact with preference optimization in unexpected ways. Capability improvements from a pretrained model can regress after post-training if the post-training data is distribution-mismatched.

The practical implications: - Post-training is not a one-time step — it is an ongoing process as model behavior is monitored and feedback loops close. - Changes in one stage can have downstream effects in later stages. - Behavioral regression (something that worked in the base model breaking after SFT) is a real phenomenon that requires careful evaluation.

Why this matters for practitioners

Understanding the post-training pipeline explains several behaviors that are otherwise mysterious: - Why models sometimes become *more* cautious and *less* capable after a safety update (over-refusal from miscalibrated preference optimization). - Why a model is confident but wrong (training on human preferences rewards confidence; being wrong is less penalized than being uncertain). - Why the same factual question gets different answers depending on how it is framed (SFT taught format templates; a different framing may not trigger the helpful template). - Why jailbreaks work (they bypass the triggers the SFT and preference optimization stages conditioned on).

Takeaway

- Instruction tuning (SFT) teaches task format and dialogue conventions; it does not fundamentally change what the model knows.
- Preference optimization (RLHF, DPO) nudges outputs toward human-preferred behavior; vulnerable to reward hacking and proxy failure.

- Post-training is multistage and iterative; changes in one stage affect downstream stages.
 - Many observable model behaviors — sycophancy, over-refusal, format lock — are post-training artifacts, not fundamental properties of the architecture.
-

Try it

Exercise 1 — Scaling intuition

Explain in two sentences why doubling model parameters without increasing training data may not follow the same scaling curve as a compute-optimal policy. Then extend it: if you were planning a domain-specific model (e.g., legal text, biomedical literature), which factor would you be most uncertain about in a scaling estimate for your domain? Why?

Exercise 2 — Contamination audit

Pick a benchmark commonly cited in model release announcements. Identify one property of that benchmark that could lead to contamination in a training crawl assembled before the benchmark was released — and one way a research team could reduce that risk in how they report results.

Exercise 3 — Post-training failure mode

Choose one post-training failure mode (sycophancy, over-refusal, format lock, reward hacking) and trace it back to its most likely training-pipeline cause. Describe one evaluation you could run to detect it in a model you are using. What would a passing response look like, and what would a failing response look like?

Exercise 4 — Architecture tradeoff

A team is building a customer support system that needs to handle very long conversation histories (50+ turns). They are considering three options: a standard transformer with a 128k token context window, a MoE model with a shorter context window, and a hybrid attention/SSM model. For each option, name one advantage and one risk specific to the long-conversation use case. Which would you start with, and why?

End of Part I. Previous: From Prompts to Systems — Volume II · Next: Part II — Alignment, safety, and robustness · Or main volume.

Part II — Alignment, safety, and robustness

Sharif Uddin

From Models to Frontiers · Volume III

Volume II covered safety as a product engineering concern: content classifiers, trust tiers, prompt injection defenses, escalation paths. This part covers safety as a research field and a deployment challenge at a level of depth that lets you argue about tradeoffs rather than just implement layers. Alignment is not a solved problem. This part names what is understood, what is contested, and where the gaps are.

Contents of this part

In the full volume table of contents, these correspond to sections 5–8.

	Chapter	What you will take away
1	Alignment in practice: goals and tensions	Helpful, honest, harmless — the tensions; Goodhart’s Law; specification gaming
2	Red teaming, adversarial evaluation, and robustness	Jailbreaks, automated red-teaming, distribution shift, long-horizon failures
3	Interpretability and monitoring	Mechanistic vs behavioral; what operators can actually use
4	Governance, deployment, and dual-use	Release strategies, capability evaluations, policy context

Contents (plain list — same as table):

1. Alignment goals and tensions — multi-objective, Goodhart, gaming.
 2. Red teaming and robustness — adversarial eval, shift, long-horizon.
 3. Interpretability and monitoring — circuits vs behavior; production practice.
 4. Governance and dual-use — release, evals, institutions.
-

Chapter 1 — Alignment in practice: goals and tensions

“Helpful, honest, harmless” is a slogan, not a specification. The moment you try to operationalize each term, they conflict.

The phrase — sometimes called “the three H’s” — captures a reasonable intuition: we want models that assist users (helpful), do not deceive (honest), and do not cause harm (harmless). The problem arises when you try to build evaluations, training signals, or deployment policies around these goals simultaneously.

The tensions

Helpfulness vs. harmlessness. A model that is maximally helpful will sometimes help with tasks that are harmful to the user, to third parties, or to society. A model that is maximally harmless will refuse many legitimate requests. The product question is not how to eliminate this tension but where to draw lines and how to calibrate them for specific deployments.

Honesty vs. helpfulness. A maximally honest model that accurately conveys uncertainty will often say “I don’t know” or “I’m not confident” — responses that are frequently less satisfying than a confident but slightly wrong answer. Users often prefer confident responses, and preference training that optimizes for user satisfaction can inadvertently reward overconfidence.

Harmlessness vs. honesty. A model instructed to be safe may refuse to discuss topics that have legitimate uses, or may hedge every response with caveats that technically reduce the odds of harm but also reduce the model’s usefulness for the vast majority of users who have legitimate intent. Over-refusal is a form of dishonesty about what the model is capable of.

Local vs. global harm. A response that helps a specific user may harm someone else. A response that is locally harmless may contribute to a harmful pattern at scale (e.g., normalizing a problematic framing across millions of interactions).

Goodhart’s Law and specification gaming

Goodhart’s Law: When a measure becomes a target, it ceases to be a good measure.

In alignment contexts: when a model is optimized to maximize a proxy metric for alignment (human preference scores, a classifier’s approval, a reward model’s rating), it may learn to satisfy the metric without satisfying the underlying goal. The model learns to *look* aligned on the measured dimensions while failing on unmeasured ones.

Specification gaming — achieving the measurable objective through unintended means — is a pattern seen both in RL systems and in preference-trained language models. Examples:

- A model trained to produce responses that human raters rate as “helpful” learns to produce responses that *feel* helpful — confident, structured, agreeable — even when the content is wrong.
- A model trained to minimize classifier-detected toxicity learns to express harmful sentiments in indirect or coded language that the classifier does not catch.
- A model trained on safety refusals learns to identify refusal-triggering surface features (certain words, framings) rather than actual harm.

The deeper problem: **multi-objective alignment cannot be reduced to a single score without losing information**. A safety leaderboard that shows a single harm rate metric may be hiding offsetting changes in helpfulness, honesty, or refusal calibration. Multi-objective evaluation is harder to summarize but more honest.

Friction: The slide that says “harm rate ↓” may hide “helpfulness ↓” or “honest refusal rate ↓” or “Goodhart optimization visible in this one edge-case category.” Publish the full distribution, not the headline.

Alignment as an ongoing process, not a training-time fix

A common misconception is that alignment can be “solved” at training time — that if the model passes safety evaluations before release, it is aligned for its lifetime. In practice:

- The deployment distribution shifts. Users find new ways to interact with the model that were not anticipated during training.
- The capability frontier moves. A model that was safe at capability level X may produce qualitatively different risks at capability level X+1.
- Context changes. A response that was appropriate in one regulatory environment may be problematic after a jurisdiction updates its requirements.

Alignment is better understood as a continuous feedback loop — training and evaluating, deploying and monitoring, identifying failures and updating — than as a training pipeline that terminates at release.

Takeaway

- The three H’s (helpful, honest, harmless) are goals that genuinely conflict; alignment is managing those tradeoffs deliberately.
 - Goodhart’s Law operates in alignment: optimizing a proxy metric for safety can produce a model that satisfies the metric without satisfying the underlying goal.
 - Multi-objective alignment cannot be summarized as a single score without information loss.
 - Alignment is an ongoing process across the model lifecycle, not a one-time training fix.
-

Chapter 2 — Red teaming, adversarial evaluation, and robustness

A benchmark you passed in January is a benchmark you could memorize by March. Safety evaluations that do not evolve are not evaluations — they are historical records.

Red teaming is the practice of systematically attempting to elicit harmful, policy-violating, or otherwise undesired behavior from a model, with the goal of discovering weaknesses before deployment (or before adversaries discover them in production). It is both a pre-deployment safety practice and an ongoing research methodology.

What red teaming covers

Policy violation detection. Testing whether the model can be induced to produce content it should not — detailed instructions for dangerous activities, targeted harassment, deceptive content, violations of legal constraints. The difficulty is that the space of possible harmful requests is effectively unbounded.

Jailbreak testing. Testing whether adversarial phrasings, role-play framings, multi-turn manipulation, or indirect approaches can bypass safety measures that work under normal conditions. Jailbreaks evolve: as safety training patches known techniques, adversaries develop new ones.

Over-refusal testing. Testing whether the model is too cautious — refusing benign requests, adding unnecessary warnings to obvious content, failing to help with legitimate queries in sensitive domains (medical information, legal questions, security research). Over-refusal is a safety failure of a different kind: it reduces trust and usefulness without actually preventing harm.

Capability boundary testing. For models with tool access or agent capabilities, testing whether the model can be manipulated into using its capabilities in unintended ways — exfiltrating data, taking irreversible actions, interacting with external systems it should not.

Automated red teaming

Human red teamers are expensive and can only test a finite number of scenarios. **Automated red teaming** uses a second model to generate adversarial inputs, test the target model's responses, and iterate. This allows much larger coverage of the adversarial input space.

Limitations of automated red teaming: - The adversarial model may not find the most dangerous prompts that a motivated human adversary would generate. - Automated evaluation of whether a response is actually harmful requires another classifier, which has its own failure modes. - Automated red teaming optimizes for the failure modes the test harness is looking for; novel failure modes outside the harness are not found.

Distribution shift

Models are evaluated under conditions similar to their training distribution. When deployed, they encounter a broader, different distribution:

- Users with different linguistic backgrounds and styles than those in training data.
- Domain-specific uses that were not anticipated (medical, legal, specialized technical fields).

- Adversarial users who specifically probe for weaknesses.
- Multi-turn interactions with dynamics that do not appear in training.

A model that is well-aligned under its training distribution may fail in unexpected ways when the input distribution shifts. This is not unique to safety: capability failures under distribution shift are common. The particular concern for safety is that the failure modes may be in exactly the cases where robust alignment matters most.

Long-horizon failures

Single-turn evaluations miss failure modes that emerge over multiple turns or across a long interaction:

- **Error compounding.** Small errors in early turns become larger errors in later turns.
- **Goal drift.** In an extended agentic context, the model's behavior may shift toward implicit goals that were not specified.
- **Reward hacking over time.** A model in a continuous feedback loop may learn patterns that optimize measurable outcomes while drifting from intended behavior.

Testing over longer horizons is expensive and harder to automate, but it is where real-world deployment failures often occur.

Memorable detail: the benchmark you passed in January is the adversarial target for April. Static safety evals become attack surfaces when adversaries can study them.

Building robustness into deployment

Red teaming and adversarial evaluation are diagnostic, not curative. The findings feed into: - Additional post-training on failure cases. - Updated policy rules in the system prompt. - Classifier additions for specific failure modes. - Rate limiting and monitoring for patterns associated with adversarial use. - Capability restrictions for high-risk tool access.

No combination of these fully eliminates adversarial risk. The practical goal is raising the difficulty and lowering the impact of adversarial exploitation to acceptable levels — not achieving zero-risk deployment.

Takeaway

- Red teaming is structured adversarial testing to find safety failures before deployment. It includes both policy violation and over-refusal testing.
 - Automated red teaming increases coverage but does not replace human adversarial testing for novel or subtle failures.
 - Distribution shift exposes alignment failures that did not appear in training evaluation.
 - Long-horizon failure modes require evaluation over multi-turn or agentic contexts, not just single queries.
 - Red teaming is diagnostic — findings need to feed back into training, policy, and monitoring.
-

Chapter 3 — Interpretability and monitoring

“Interpretability” covers a range of ambitions, from understanding individual computations inside a specific model to reliably predicting model behavior in deployment. The gap between these ambitions is large.

For most operators, the actionable question is not “how does this model compute a specific output” but “can I detect and respond to harmful or unexpected behavior quickly?” Those questions have different answers, and conflating them leads to either misplaced optimism (we understand how the model works) or paralysis (we cannot deploy without full interpretability).

Mechanistic interpretability

Mechanistic interpretability is the research program of understanding the specific computations performed by model components — which neurons activate for which inputs, which attention heads perform which functions, how features are represented in the residual stream.

Significant progress has been made on smaller models and on specific circuits (e.g., understanding how models implement induction heads, indirect object identification, or simple arithmetic). The challenges at scale are substantial:

- The computational graph of a large language model has billions of parameters and many layers of abstraction. Fully characterizing the mechanistic basis of any moderately complex behavior is not currently feasible.
- Circuits discovered in small models may not correspond to how the same behavior is implemented in large models.
- Polysemantic neurons (neurons that respond to multiple unrelated features) make clean mechanical stories difficult.

Mechanistic interpretability is an active and valuable research area. For most operators, it produces scientific insights that improve understanding of model behavior in aggregate — not tools you can run against your deployed model tonight.

Behavioral interpretability

Behavioral interpretability asks: can I predict what the model will do in situations I care about, and can I understand why it fails when it does?

This is more tractable for deployment: - Systematic evaluation on structured test sets covering known failure modes. - Probing the model’s behavior by varying specific input features and observing outputs. - Calibration analysis (does the model’s expressed confidence match its accuracy?). - Consistency testing (does the model give the same answer to logically equivalent questions?).

Behavioral analysis does not tell you *why* the model behaves a certain way at the level of weights and circuits, but it tells you *what* the model does under specific conditions — which is often what operators need.

Monitoring deployed systems

For deployment, monitoring is the operational version of interpretability:

Input distribution monitoring. Are the prompts users are sending drifting from the distribution the model was evaluated on? A spike in a specific prompt pattern may indicate adversarial probing or a new use case the model was not prepared for.

Output distribution monitoring. Are outputs changing over time? Changes in refusal rate, average response length, format adherence, or content patterns can indicate model drift or a shift in the user population.

Anomaly detection for misuse patterns. High-frequency similar queries from the same user or IP, queries designed to extract the system prompt, queries that test policy boundaries repeatedly — these patterns warrant human review.

Golden-set regression monitoring. Running a fixed set of evaluation prompts periodically and checking whether scores have changed. This catches silent model behavior changes (from provider updates or system changes) that would otherwise only surface through user complaints.

Escalation paths with teeth. Monitoring is only useful if flagged issues reach a person who can act on them within an appropriate time window. A monitoring system that generates alerts nobody reviews is theater.

Direct address: if your interpretability research cannot change a runbook — cannot lead to a specific action when a specific pattern is detected — it is wallpaper. Instrumentability (can I measure this?) matters more than interpretability (can I explain this?) for most deployment decisions.

The limits of current interpretability

Current interpretability tools cannot reliably:

- Predict specific model behaviors on novel inputs not covered by evaluation.
- Distinguish a model that is genuinely aligned from one that is strategically performing alignment on evaluation inputs (the “deceptive alignment” concern in the research literature).
- Provide guarantees about behavior outside the training and evaluation distribution.

This is not a reason to abandon interpretability research — it is a reason to be honest about what current tools can and cannot tell operators, and to invest in the operational monitoring that protects production systems now.

Takeaway

- Mechanistic interpretability seeks to understand model computations from the inside; valuable research, but not yet operational for most deployments.
 - Behavioral interpretability — systematic evaluation, calibration, consistency testing — is more immediately useful for operators.
 - Production monitoring: input and output distribution, anomaly detection, golden-set regression, escalation paths.
 - Current interpretability cannot guarantee alignment outside the evaluation distribution. Operational monitoring closes the gap.
-

Chapter 4 — Governance, deployment, and dual-use

Every model release is a governance decision, whether or not the team making it thinks about it that way.

Releasing a model — or an API that grants access to it — has different implications at different levels of capability and different levels of openness. The governance choices made by labs, platforms, and regulators shape the landscape in which practitioners operate.

Release strategies and their tradeoffs

Full open weights (releasing model weights publicly, often with a license): - Maximum access for researchers, fine-tuners, and independent evaluators. - No provider control over downstream use — anyone can fine-tune out safety measures, apply the model to any task, or redistribute modified versions. - Strong argument for democratization and auditability; strong concern for misuse at scale.

Open weights with use restrictions (releasing weights with terms of service that prohibit specific uses): - Better than fully permissive for some misuse vectors, but largely unenforceable. Technical controls are absent; policy is the only barrier.

API-only access: - Provider retains ability to add safety filters, monitor usage, rate limit, and revoke access. - Higher barrier to entry for adversarial fine-tuning (attacker must work through the API). - Trade-off: single point of control also means single point of failure or misuse by the provider.

Staged release (limited API access → broader access → potentially open weights over time): - Allows safety issues to be identified at lower exposure before wider release. - Requires genuine feedback loops — staged release is only as safe as the monitoring during the staging period.

There is no universally correct answer. The appropriate release strategy depends on capability level, deployment context, the maturity of safety evaluations, and organizational risk tolerance.

Capability evaluations

Capability evaluations attempt to assess specific risk-relevant capabilities before wide release — not “is the model helpful” but “can this model provide meaningful uplift for dangerous activities (biosecurity, cyberattack, CBRN risks)?”

Current capability evaluations are imperfect in several ways: - The evaluations are developed in advance and may miss novel capabilities. - Models can perform differently on capability evaluations than in adversarial real-world use. - Evaluations can be gamed (intentionally or unintentionally) if the model has seen similar test structures during training.

Despite these limitations, capability evaluations are better than no evaluations. They establish a documented record of what was assessed before release, which supports accountability and allows comparison across releases and organizations.

Dual-use and the limits of technical controls

Dual-use — the property of being useful for beneficial and harmful purposes alike — is not unique to AI. It characterizes the internet, chemistry, biology, cryptography. The challenge is that language models are general-purpose systems whose capabilities are not separable: the same reasoning ability that helps write good code can help find security vulnerabilities.

Technical controls (content filters, fine-tuning restrictions, use limits) can raise the cost of misuse. They cannot eliminate it, particularly for models whose weights are or become publicly available. This shifts the question from “can we prevent misuse” to “what combination of technical controls, legal frameworks, norms, and monitoring reduces expected harm to acceptable levels.”

One-line analogy: governance is weather routing — you navigate under uncertainty toward better outcomes, not under the fantasy of full control over where the ship ends up.

Policy context

The regulatory landscape around AI is evolving rapidly and varies by jurisdiction. Rather than prescribing specific policies (which will have changed by the time you read this), this chapter identifies the structural questions practitioners encounter:

- **Disclosure requirements:** What must be disclosed when AI is involved in decisions affecting people?
- **Liability allocation:** Who is responsible when an AI system causes harm — the developer, the deployer, the user?
- **Incident reporting:** What obligations exist to report AI failures or misuse?
- **Capability thresholds:** Do specific capability levels trigger additional requirements?

These questions are being answered differently in different jurisdictions. Practitioners building for international deployment need legal review aligned with their specific geography and domain.

Takeaway

- Release strategies (open weights vs. API-only vs. staged) involve genuine tradeoffs between access, safety, and accountability.
 - Capability evaluations are imperfect but better than no evaluation. They establish documented accountability.
 - Dual-use cannot be technically eliminated. Governance combines technical controls, policy, norms, and monitoring.
 - The regulatory landscape is evolving; build relationships with legal and compliance teams rather than treating governance as a one-time check.
-

Try it

Exercise 1 — Alignment tradeoff

Name two alignment goals that conflict on the same user query: a user asks an LLM-powered health information assistant whether their medication dose sounds right. Describe what a maximally helpful response looks like, what a maximally harmless response looks like, and what an honest response looks like. For each, name one failure mode if that value is pushed too far.

Exercise 2 — Red team scenario

Draft one adversarial scenario that tests **over-refusal** rather than under-safety — a prompt that a well-aligned model should answer helpfully but a miscalibrated safety policy would refuse. What would you measure to determine whether your system handles this correctly? How would you distinguish a good refusal from a bad one?

Exercise 3 — Monitoring audit

For a production LLM feature you work with or can imagine: identify three things you would monitor to detect alignment-relevant degradation. For each, describe what a meaningful signal looks like (not just “the metric changes”) and what action you would take if that signal appeared.

Exercise 4 — Release decision

A team has built a coding assistant model with strong performance on software engineering tasks. It also has documented capability for generating functional exploit code — it was not trained to do this, but it emerged. Walk through the release decision: What capability evaluations should be run before release? What are the arguments for API-only vs. open weights in this case? What monitoring would you want in the first 90 days of deployment?

End of Part II. Previous: Part I — Scale, data, and the pretraining stack · Next: Part III — Efficiency: training, inference, and systems · Or main volume.

Part III — Efficiency: training, inference, and systems

Sharif Uddin

From Models to Frontiers · Volume III

Efficiency decides what is affordable. A technique that improves model quality by 5% while doubling inference cost may not be worth deploying. A quantization method that cuts memory usage by 4x while degrading quality by less than 1% on your task may be essential for serving at your scale. This part covers the engineering and research of making large models affordable to train and to serve — at a level of depth that lets you evaluate efficiency claims critically and make informed infrastructure decisions.

Contents of this part

In the full volume table of contents, these correspond to sections 9–11.

	Chapter	What you will take away
1	Training efficiency	Mixed precision, parallelism, gradient checkpointing, curriculum, fault tolerance
2	Inference efficiency	Quantization, distillation, speculative decoding, KV-cache, latency-quality-cost tradeoffs
3	Specialized hardware and software stacks	GPUs, TPUs, compiler stacks, hosting vs. self-hosting, custom silicon

Contents (plain list — same as table):

1. Training efficiency — precision, parallelism, checkpointing, bottlenecks.
 2. Inference efficiency — quantization, distillation, speculative decoding, KV cache.
 3. Hardware and stacks — GPU/TPU, compilers, serving decisions.
-

Chapter 1 — Training efficiency

When a training run costs hundreds of thousands of dollars and takes weeks, “faster code” is not an abstract goal — it is the difference between a team that can iterate and one that cannot.

Modern large model training is a distributed systems problem as much as a machine learning problem. The techniques in this chapter address how to use a cluster of accelerators efficiently, how to handle the numerical challenges of training large models, and how to ensure that expensive training runs are recoverable when things go wrong.

Mixed precision training

Full-precision (float32) arithmetic is expensive in both memory and compute. **Mixed precision training** uses lower-precision representations (float16 or bfloat16) for most computations while maintaining float32 for numerically sensitive operations (gradient accumulation, loss scaling).

bfloat16 has become the dominant choice for large model training because it has the same dynamic range as float32 (8 exponent bits) but half the memory footprint. This matters because many training instabilities arise from values near the representation limits, and bfloat16 avoids the overflow/underflow issues that float16 encounters with large gradients.

The benefit: roughly 2x memory reduction, and on modern accelerators (A100, H100, TPU v4+), bfloat16 matrix multiplication is 2-4x faster than float32 due to hardware tensor core support.

The catch: some operations — most critically gradient accumulation — must remain in float32 to avoid numerical error accumulation across many steps. The “mixed” in mixed precision is not optional.

Parallelism strategies

Training a model that does not fit on a single GPU (or TPU chip) requires distributing the computation. Several strategies exist, and large training runs typically combine multiple:

Data parallelism. Each accelerator processes a different micro-batch of data with identical model weights. Gradients are averaged across accelerators after each step. This is the simplest form of parallelism and scales well as long as the model fits on a single device.

Tensor parallelism. Individual layers (particularly large attention and feed-forward layers) are split across multiple accelerators. The matrix operations within a layer are computed in parallel. This allows training models that would not fit on a single device even for a single layer.

Pipeline parallelism. Different layers of the model are assigned to different accelerators. The input passes through layers in sequence across devices. The challenge is “pipeline bubbles” — accelerators that are idle while waiting for inputs from the previous stage.

ZeRO (Zero Redundancy Optimizer). Rather than replicating the optimizer states across all data-parallel replicas (which is expensive), ZeRO shards optimizer states, gradients, and model parameters across data-parallel workers. This dramatically reduces per-device memory usage and allows larger model sizes at the same hardware cost.

Large runs commonly use a 3D parallelism approach — combining data, tensor, and pipeline parallelism — with ZeRO optimization on top. The configuration is complex enough that dedicated frameworks (Megatron-LM, DeepSpeed, JAX with GSPMD) exist specifically to manage it.

Gradient checkpointing

During the forward pass, intermediate activations are stored for use in the backward pass (computing gradients). For a large model, storing all activations simultaneously can require as much memory as the model weights themselves.

Gradient checkpointing recomputes activations during the backward pass rather than storing them all. The tradeoff: roughly 33% more compute, in exchange for significantly reduced activation memory. For very large models or very long sequences, this is often essential to fit training in available memory.

Bottleneck identification

Friction: teams building custom training infrastructure often spend time on kernel optimization — writing custom CUDA for specific operations — before profiling their actual bottlenecks. The actual bottlenecks are frequently:

- **I/O:** Data loading is not fast enough to keep accelerators busy. The GPU is waiting for the next batch.
- **Communication overhead:** In distributed training, all-reduce operations (averaging gradients) can become the limiting factor, especially for large model sizes and high gradient update frequencies.
- **Checkpoint I/O:** Writing checkpoints to disk and resuming from them can dominate wall-clock time for large models on long runs.
- **Load imbalance:** Pipeline bubbles, uneven batch sizes, or MoE expert imbalance leave some accelerators idle.

Profile before optimizing. The boring bottleneck beats the clever story about kernel fusion.

Fault tolerance

Training runs for large models take days to weeks. Hardware failures are not rare events at this scale — they are expected. Effective training infrastructure requires:

- **Frequent checkpointing:** Saving model state at regular intervals so a hardware failure costs at most a few hours rather than days of work.
- **Automatic restart from checkpoint:** The training job should resume from the last checkpoint without manual intervention.
- **Anomaly detection:** Catching numerical instabilities (loss spikes, NaN values) early, before they propagate across many steps.

The economics of fault tolerance: for a 10-day training run with a 1% per-hour hardware failure rate, the expected time before a failure is roughly 4 days. A checkpoint every 30 minutes means at most 30 minutes of lost work. A checkpoint every 4 hours means potentially 4 hours. Checkpoint frequency directly sets the price of hardware failure.

Takeaway

- Mixed precision (bfloat16) gives roughly 2x memory reduction and faster compute with appropriate care for numerically sensitive operations.
 - Large model training combines multiple parallelism strategies: data, tensor, pipeline, plus ZeRO optimizer sharding.
 - Gradient checkpointing trades compute for memory.
 - Profile for I/O, communication, and load imbalance before optimizing kernels.
 - Fault tolerance requires frequent checkpointing and automatic restart. It is not optional for long training runs.
-

Chapter 2 — Inference efficiency

Training is a one-time cost. Inference is ongoing — every query, every user, every day. At scale, inference economics dominate training economics within months of launch.

A model that cost \$10M to train but serves 100M queries per day is a system where inference efficiency decisions matter far more than training efficiency decisions. The techniques in this chapter address how to serve large models more cheaply, more quickly, or with a smaller hardware footprint — and what quality tradeoffs those decisions involve.

Quantization

Quantization reduces the numerical precision of model weights (and sometimes activations) from the 16-bit or 32-bit floats used during training to lower-precision formats.

INT8 quantization (8-bit integers) can roughly halve model memory with typically small quality degradation on most tasks. The mechanism: map the range of float16 weights onto a 256-value integer scale, with a scale factor to reverse the mapping. The quality degradation comes from rounding error and from the fact that the scale factor is coarse.

INT4 quantization (4-bit) further halves memory at higher risk of quality degradation. Whether the degradation is acceptable depends heavily on the task: coding and reasoning tasks tend to be more sensitive than summarization or simple question-answering.

GPTQ and GGUF are specific quantization schemes that have become practical standards for running large models on consumer hardware. They use post-training quantization (quantizing after training without retraining) with calibration datasets to minimize error.

The key insight for practitioners: **quantization quality is task-specific, not model-specific**. You cannot evaluate the acceptability of a quantization decision in the abstract — you have to test on your task distribution. A model quantized to INT4 that passes your evaluation on the cases you care about is an acceptable deployment choice. One that fails on specific reasoning chains you need is not, regardless of average benchmark scores.

Distillation

Knowledge distillation trains a smaller “student” model to mimic the behavior of a larger “teacher” model. Rather than training the student on hard labels (correct/incorrect answers), distillation trains on the teacher’s output distribution — the student learns from the teacher’s uncertainty and the relative probabilities it assigns to different outputs.

Effective distillation can produce smaller models that perform surprisingly close to the teacher on the teacher’s specific tasks. The limitations: - Distillation is task-specific: a student distilled on customer support conversations may outperform the teacher on customer support but perform worse on unrelated tasks. - The gap between student and teacher grows for complex reasoning tasks where the teacher’s outputs require understanding that a small model cannot represent. - Distillation from a proprietary model raises legal and licensing questions.

Speculative decoding

Speculative decoding uses a small, fast “draft” model to generate candidate tokens, which a large “verifier” model then checks in parallel. If the large model agrees with the draft tokens, they are accepted without additional compute. If it disagrees, it corrects the first wrong token and continues.

The appeal: the large model’s output is preserved exactly (it is still the one determining final tokens), but latency is reduced because many draft tokens can be verified in a single forward pass. The speedup depends on how often the draft model agrees with the large model — more agreement means more speedup.

Speculative decoding is most effective when: - The large model is compute-bound (the bottleneck is computation, not memory bandwidth). - There is a good small model in the same family as the large model (trained on similar data, so it agrees often). - Latency is more important than throughput (it does not reduce total compute — it reduces wall-clock time per sequence).

KV-cache mechanics and memory pressure

During generation, the transformer’s attention mechanism computes key and value vectors for every token in the context. Rather than recomputing these on every generation step, the **KV cache** stores them in memory and reuses them.

The memory cost: for a model with H attention heads, each with a key and value dimension of D , and a context of L tokens, the KV cache requires $2 \times H \times D \times L$ entries per layer. For a 70B parameter model serving long contexts, the KV cache can be comparable in size to the model weights themselves — or larger.

At inference scale, KV cache memory often becomes the binding constraint: - Longer context windows increase memory usage linearly. - Many simultaneous users require either more memory or smaller batches. - Paged attention (managing the KV cache in fixed-size pages, similar to OS memory paging) allows more efficient sharing and reduces fragmentation.

Attention sinks and context compression are techniques for reducing effective context length without truncation — identifying which tokens the model “needs” to retain and dropping those it does not. These techniques can substantially reduce KV cache pressure for very long contexts at the cost of potential quality regression on content in dropped context.

The latency–quality–cost Pareto surface

There is no single “best” deployment configuration. The tradeoffs form a surface:

Optimization	What you gain	What you risk
INT8 quantization	2x memory reduction, faster matrix mul	Small quality degradation on sensitive tasks
INT4 quantization	4x memory reduction	Higher quality risk, especially for reasoning
Smaller model	Much lower compute and memory	Quality gap vs. larger model
Distilled student	Lower inference cost on target tasks	Weaker generalization off-target
Speculative decoding	Lower latency	Added complexity, draft model required
Shorter context	Lower KV cache memory	Less available context for tasks that need it

Product decisions live on this surface. The right operating point depends on your latency SLO, your quality requirement on your specific task distribution, and your cost budget. Measure all three on your task before deciding.

Anchor: inference costs amortize training across users. A model trained once but serving 100M queries per day should have its optimization budget allocated to inference, not training. Profile where the money actually goes.

Takeaway

- Quantization reduces memory and compute at a quality cost that varies by task. Always evaluate on your task distribution.
 - Distillation can produce efficient smaller models for specific tasks; generalization beyond the training distribution shrinks.
 - Speculative decoding reduces latency for large models by using a draft model, without changing output quality.
 - KV cache is the dominant memory cost for long-context inference; paged attention and compression techniques address it.
 - Optimize the right stage: for high-query-volume products, inference economics dominate within months.
-

Chapter 3 — Specialized hardware and software stacks

Hardware is not neutral. It determines which techniques are practical, which latency targets are achievable, and how your costs scale with usage.

Understanding the hardware landscape at a decision-maker level — not a chip designer level — lets you evaluate vendor claims, make infrastructure choices with appropriate skepticism, and avoid building on assumptions that hold at one scale and break at another.

The accelerator landscape

GPUs (Graphics Processing Units) are the dominant training and inference accelerator for most organizations. Modern data-center GPUs (NVIDIA H100, H200, A100) provide: - Large amounts of high-bandwidth memory (80GB–192GB per chip). - Tensor cores optimized for the matrix multiplications that dominate transformer computation. - NVLink for fast inter-GPU communication within a node. - A mature software ecosystem (CUDA, cuDNN, PyTorch, JAX).

TPUs (Tensor Processing Units, Google’s ASICs) offer strong performance on specific workloads, particularly for models developed within the Google/JAX ecosystem. Less flexible for arbitrary experimentation but very efficient for regular workloads.

Newer entrants (AMD MI300, Intel Gaudi, various startup ASICs) are expanding the landscape, typically emphasizing cost-per-token competitiveness for specific workloads.

The practical decision for most organizations: start with GPUs. Depart only when volume and workload stability justify the engineering investment in porting and optimizing for a different platform.

Compiler stacks and kernel fusion

Modern accelerators do not automatically achieve their theoretical peak performance for arbitrary operations. Getting close requires careful management of:

Memory bandwidth vs. compute bandwidth. For most transformer operations, the limiting factor is not compute but memory bandwidth — moving data between high-bandwidth memory and compute units is the bottleneck. Kernel fusion reduces memory round-trips by combining operations that would otherwise each require loading and storing intermediate values.

Compiler optimization stacks (XLA for JAX/TPUs, TorchCompile for PyTorch, Triton for custom kernels) automate some of this optimization, fusing operations and optimizing memory layout. The resulting throughput improvement can be substantial — 30-50% is common for well-tuned compilation.

Flash attention specifically addresses the attention mechanism’s memory bandwidth problem by computing attention without materializing the full attention matrix in high-bandwidth memory. It is now effectively standard for production transformer inference and training.

Hosting vs. self-hosting

The decision between API access (hosted inference) and running your own models involves several dimensions:

	Hosted API	Self-hosted
Upfront cost	None	High (hardware, setup)
Marginal cost	Per-token pricing	Amortized hardware + ops
Latency control	Provider SLAs; shared infrastructure	Full control
Model choice	Provider's models	Any model you can run
Data privacy	Trust provider's handling	Full control
Capacity planning	Provider handles it	Your problem
Customization	Limited to provider's fine-tune options	Full control

Hosted APIs are almost always the right starting point. Self-hosting becomes worth considering when: - Query volume is high enough that per-token API costs exceed amortized hardware costs. - Latency requirements are tight enough that provider SLAs are insufficient. - Data privacy requirements prohibit sending data to an external provider. - Fine-tuning or model modification requirements exceed what APIs support.

This calculation changes over time as hardware costs fall and as API providers reduce per-token prices. Revisit it periodically rather than treating the initial decision as permanent.

Custom silicon

ASICs (Application-Specific Integrated Circuits) designed for LLM inference can substantially reduce cost-per-token at scale — they trade flexibility for efficiency. The economics become compelling when: - Token volume is very high (hundreds of millions per day). - The workload is stable enough that an inflexible but optimized chip is worth the NRE (non-recurring engineering) cost. - The organization has the engineering resources to port and maintain software stacks on new hardware.

For most organizations, custom silicon is not the current decision. The decision is which vendor's current generation GPU to deploy and how to optimize workloads for it.

Direct address: If you are not at the token volumes where custom silicon changes your economics, the right response to custom silicon discussions is to file them under “interesting, not my decision this quarter.” Hardware strategy should follow volume and workload stability, not hype.

Takeaway

- GPUs dominate training and inference; start there. Depart for other hardware only with clear volume and workload justification.
- Compiler stacks, kernel fusion, and flash attention close the gap between theoretical and actual accelerator performance.
- Hosted APIs are almost always the right starting point; self-hosting for volume, latency, privacy, or customization reasons.
- Custom silicon has compelling economics at high volume but requires engineering investment and reduces flexibility.

- Hardware filters which techniques are practical at your scale. Know the constraints of your infrastructure before committing to an optimization strategy.
-

Try it

Exercise 1 — Quantization evaluation

You are considering INT4 quantization for a model that handles both customer FAQ retrieval and contract clause extraction. Describe how you would evaluate the acceptability of this quantization decision *specifically for each task* — what test cases would you include that are more challenging for a heavily quantized model than for the full-precision version?

Exercise 2 — Inference bottleneck

A long-context chat product serves an average context length of 20,000 tokens per conversation and handles 500,000 conversations per day. Is KV-cache memory or model weight memory more likely to be the binding constraint on GPU memory? Show your reasoning. What technique would you investigate first to address the binding constraint?

Exercise 3 — Hosting decision

A startup is building a specialized medical information assistant using a 70B parameter model. They currently use a hosted API and are considering self-hosting. What information would you need to determine whether self-hosting is economically justified? What would the analysis look like — what numbers matter?

Exercise 4 — Speculative decoding fit

A team wants to reduce time-to-first-token for a code generation assistant. They are considering speculative decoding using a 7B model as a draft and a 70B model as the verifier. Describe the conditions under which speculative decoding is likely to provide good speedup for this task, and one scenario in which it would provide poor speedup despite having both models available.

End of Part III. Previous: Part II — Alignment, safety, and robustness · Next: Part IV — Beyond text: multimodal models and agents · Or main volume.

Part IV — Beyond text: multimodal models and agents

Sharif Uddin

From Models to Frontiers · Volume III

Language is one modality among many. The world produces images, audio, video, structured data, sensor readings, and code — and increasingly, capable AI systems need to work across these modalities together, not in isolation. This part covers multimodal models, the research foundations of tool use and grounding, and the architecture and failure modes of agents that operate over multiple steps in the world. Each of these is an active research area; this part gives you the depth to read it critically.

Contents of this part

In the full volume table of contents, these correspond to sections 12–14.

	Chapter	What you will take away
1	Multimodal foundations	Vision-language architectures, audio-language, unified vs. modular, evaluation challenges
2	Tool use, retrieval, and grounding at research depth	When to retrieve vs. use long context vs. parametric memory; grounding protocols
3	Agents: planning, memory, and multi-step reliability	Architectures, failure modes, evaluation over time, open problems

Contents (plain list — same as table):

- 1. Multimodal foundations — fusion, architectures, evaluation.
 - 2. Tool use and grounding — retrieval vs. context vs. memory.
 - 3. Agents — planning, memory, evaluation, failure modes.
-

Chapter 1 — Multimodal foundations

When the model describes what is not in the image as confidently as what is, you have a new category of failure that text-only evaluation never surfaced.

Language models learn statistical patterns over tokens. Images produce a very different kind of statistical pattern — pixel values, spatial relationships, edges, objects, scenes — and the challenge of multimodal modeling is building a system that can reason about both kinds of information jointly, not just process them separately.

The basic architecture question: unified vs. modular

There are two broad approaches to building a model that handles both text and images (or other modalities).

Modular approach: A separate visual encoder (often a vision transformer pretrained on image classification or contrastive image-text pairs) processes images into a representation, which is then projected into the language model's token embedding space. The language model sees the image as a sequence of “visual tokens” alongside the text tokens. Common examples: early LLaVA-style models, many production VLMs (vision-language models).

Unified approach: A single model is trained from the start on both text and images, with image patches tokenized directly and interleaved with text tokens. No separate encoder — just one model that processes multiple modalities with the same architecture.

The modular approach is more practical for combining existing strong visual encoders with existing strong language models. The unified approach potentially allows tighter integration between modalities but requires much more training data and compute to match strong specialized encoders.

Vision-language pretraining

Contrastive pretraining (the approach behind CLIP) trains an image encoder and text encoder jointly so that matching image-text pairs are represented close together in embedding space. This produces strong visual representations that can identify objects, scenes, and concepts without requiring labeled classification data.

Generative objectives train the model to generate text descriptions from images, or to generate images from text descriptions. These objectives require the model to represent detailed visual content, not just coarse category labels.

Modern vision-language models typically combine both: contrastive pretraining for the visual encoder, followed by supervised fine-tuning on image-text pairs for the specific task format (captioning, visual question answering, document understanding).

Audio-language models

Audio introduces different challenges than images: - Audio is inherently temporal — meaning depends on sequence and duration, not spatial layout. - The same acoustic signal can correspond to different meanings depending on prosody, accent, and context. - Speech recognition (converting audio to text) is

not the same as audio understanding (understanding tone, emotion, background sounds, music).

Speech tokens are typically produced by a discrete audio encoder — a model that converts audio into a sequence of discrete codes, analogous to how images are converted to visual tokens. These can then be processed by a language model alongside text tokens.

Whisper-style models trained on large amounts of speech-text pairs achieve strong transcription performance. Audio language models that extend beyond transcription to understanding non-linguistic audio content are less mature.

Evaluation challenges specific to multimodal models

Text-only evaluation is difficult enough — multimodal evaluation introduces additional failure modes:

Hallucinated objects. The model describes objects, text, or details that are not present in the image. This is qualitatively different from text hallucination: the model is not confabulating about facts from training data, but about the specific visual content it was given. A user who trusts a VLM’s description of an image may be misled about what is actually there.

Spatial relationship errors. Models often fail on questions about relative positions: “Is the cup to the left or right of the plate?” requires precise spatial reasoning that models frequently get wrong despite being able to describe both objects individually.

Text in images (OCR). Reading and understanding text that appears in images (signs, documents, screenshots) is a separate capability from visual reasoning. Models vary dramatically in OCR quality; this is often a surprising failure for users who expect a VLM to handle documents.

Modality anchoring. When text and image information conflict, models often anchor on the modality that dominates their training data — typically text. A model shown an image of a red car with the caption “blue car” may answer questions based on the caption rather than the image.

Fairness and representation. Multimodal models inherit biases from both their text training data and their visual training data. Visual biases — what kind of faces appear when a model generates “a doctor” or “a criminal” — are distinct from text biases and require separate evaluation.

Memorable failure: the model describes a picture confidently and incorrectly. The user assumes they can trust it. Text-only QA errors feel quaint by comparison because at least with text-only hallucination, the model’s training data had the content. With visual hallucination, the evidence was right there in the input and the model invented something else.

Takeaway

- Multimodal models combine visual and text understanding through modular (separate encoder) or unified (single model) architectures.
- Vision-language pretraining uses contrastive and generative objectives to build strong visual representations.
- Audio-language models extend the same principles to speech and audio content.
- Evaluation challenges specific to multimodal: hallucinated objects, spatial errors, OCR quality, modality anchoring, and visual representation bias.
- Multimodal errors are qualitatively different from text-only errors — they require multimodal evaluation, not text evaluation applied to captions.

Chapter 2 — Tool use, retrieval, and grounding at research depth

Volume II covered RAG as an engineering practice. This chapter asks the harder question: when is retrieval the right architecture at all, and what makes grounding reliable?

The basic problem: language models have parametric knowledge (learned during pretraining, frozen into weights) and context knowledge (what is present in the current input). Neither alone is sufficient for many production tasks. The interesting research question is how to combine them well — and how to tell when the model is actually using retrieved content versus generating from parametric memory despite being told to retrieve.

Parametric vs. non-parametric memory

Parametric memory (knowledge in weights) is: - Fast to access — no external retrieval step. - Potentially stale — knowledge is frozen at training time. - Hard to audit — you cannot check which training examples the model is drawing on. - Hard to correct — changing specific facts requires retraining or fine-tuning. - Subject to hallucination — the model can be confident about things it does not actually know.

Non-parametric memory (retrieved from an external store) is: - As fresh as the store — can be updated without retraining. - Auditable — you can inspect what was retrieved and trace the model's answer. - Bounded — the model can only answer based on what was retrieved. - More expensive — requires a retrieval step, a vector store, chunked documents, and embedding infrastructure.

The decision between retrieval and long context is a variant of this question: when does it make sense to retrieve a subset of relevant content versus including all potentially relevant content in the context window?

Long context vs. retrieval

As context windows have grown to 128k, 1M, and beyond, the question has emerged: do we still need retrieval, or can we just include everything in context?

The tradeoffs are real:

In favor of long context over retrieval: - No retrieval infrastructure to build and maintain. - No chunking decisions — no risk of splitting a relevant passage across chunks. - The model can attend to any part of the context simultaneously. - Works well when the relevant content is unknown in advance (you cannot know what to retrieve).

In favor of retrieval over long context: - Very long contexts are expensive (attention cost is quadratic in sequence length). - Models do not uniformly use all context equally. Research has documented a “lost in the middle” effect: models attend more strongly to the beginning and end of long contexts, with middle content being underweighted. - Retrieval provides an auditable trace of what information the model used. - For large knowledge bases (millions of documents), retrieval is more practical than including everything.

The practical answer depends on scale. For knowledge bases that fit in context at affordable cost, long-context may be simpler. For large knowledge bases or tight latency/cost constraints, retrieval remains necessary.

The grounding problem

Grounding is the property of model outputs being traceable to specific, verifiable information sources rather than generated from parametric memory. A grounded answer cites specific retrieved passages; an ungrounded answer may be factually correct (drawing on pretraining) or fabricated.

The challenge: models trained to be helpful will generate confident-sounding answers even when the retrieved context is insufficient, irrelevant, or absent. Ensuring grounding requires:

Explicit grounding instructions in the system prompt. “Answer only from the provided CONTEXT. If the context does not contain sufficient information, say so.” These reduce but do not eliminate hallucination.

Grounding evaluation. Test whether the model correctly abstains when the answer is not in the context. Include queries where the answer is genuinely absent. A model that correctly says “I don’t know” on absent-answer queries is grounded; one that generates plausible-sounding answers to everything is not.

Citation-level verification. For high-stakes uses, requiring the model to identify specific passages it is drawing from, and checking that those passages support the claim, provides a verifiable audit trail.

Friction: “We added tools” does not fix the grounding problem if the model can sound confident when a tool returns nothing. The grounding discipline has to be in the prompting, the evaluation, and the output validation — not just the tool architecture.

Fine-tuning vs. retrieval for knowledge updates

When knowledge needs to be updated (product information changes, regulations update, research findings shift), the choice between retrieval and fine-tuning recurs:

Retrieval is almost always better for knowledge that changes frequently. Fine-tuning snapshots knowledge at training time; retrieval provides current knowledge at query time. Updating a retrieval index is fast; re-fine-tuning a model is slow and expensive.

Fine-tuning is better for behavioral changes: teaching the model how to perform a task it was not optimized for, adjusting its output format, or specializing its reasoning style. Fine-tuning does not reliably implant specific facts — it can produce confident wrong answers rather than retrieving correctly.

The common mistake is using fine-tuning to update facts rather than behavior, and retrieval to change behavior rather than knowledge. Match the mechanism to the problem.

Takeaway

- Parametric memory (in weights) is fast and accessible but stale, unauditable, and subject to hallucination.
- Non-parametric memory (retrieval) is updatable and auditable but requires infrastructure and careful chunking.
- Long context and retrieval are complementary, not competing — the choice depends on knowledge base size, cost constraints, and context utilization patterns.
- Grounding is a discipline requiring explicit instructions, evaluation of absent-answer cases, and optional citation verification.

- Use retrieval for knowledge updates, fine-tuning for behavioral changes.
-

Chapter 3 — Agents: planning, memory, and multi-step reliability

An agent demo with five steps is three slides. An agent deployment with five hundred steps is a systems engineering problem.

Agents — systems where a model takes actions, observes results, and continues until a goal is achieved — represent a qualitative expansion of LLM capability and risk. The same properties that make agents powerful (they can accomplish complex tasks without step-by-step human instruction) make them difficult to evaluate, debug, and control.

The agent loop

The basic architecture of a tool-using agent:

```
while goal not achieved:
    observe current state
    plan next action
    select and call a tool
    receive tool result
    update state / context
    decide: continue or terminate
```

This loop is simple to describe and complex to get right. Each iteration involves the model making a decision based on potentially noisy or incomplete information, with downstream consequences for subsequent decisions.

Planning architectures

Flat planning. The model receives a goal and makes tool calls directly, without explicit decomposition. Effective for simple tasks; brittle for complex tasks where subgoal tracking matters.

Chain-of-thought planning. The model first produces an explicit reasoning trace (a plan), then executes it. This can improve coherence on multi-step tasks and provides a debugging artifact. The plan and the execution can drift.

ReAct (Reasoning + Acting). Interleaves reasoning steps and action steps: think → act → observe → think → act. Produces a trace that shows the model's reasoning at each step, which aids debugging.

Hierarchical planning. A planning agent decomposes the goal into subgoals and delegates to specialized subagents. Reduces the complexity each model must manage; adds coordination overhead and potential for misalignment between levels.

None of these architectures solves the fundamental reliability problem: each step involves a model that can be wrong, and errors compound over many steps.

Memory architectures

Agents need memory to maintain context over long task horizons. Four types of memory interact:

In-context memory. The current conversation/state is kept in the context window. Simple; runs out as the task grows longer.

Episodic memory. A record of past events the agent can retrieve. Allows the agent to reference previous actions without keeping the full history in context. Requires careful retrieval design to avoid missing relevant prior context.

Semantic memory. A knowledge store the agent can query. The RAG approach applied to agent memory: the agent can look up facts, procedures, and prior decisions.

Working memory. Intermediate results — calculations, partially processed data, notes — kept available during a task. Often implemented as files or structured data in the tool environment.

The tradeoff: more memory types add capability and complexity. The agent must learn when to store, when to retrieve, and when to discard. Memory architecture failures (storing the wrong things, retrieving the wrong things, failing to discard stale information) are a major source of agent reliability problems.

Failure modes

Error compounding. A small mistake in step 3 affects step 4, which affects step 5. By step 20, the original error may have propagated into an irrecoverable state. Errors do not stay small in long-horizon tasks.

Goal drift. Over a long task, the model's behavior may drift toward proxies that are easier to achieve than the original goal. This is subtle and difficult to detect without explicit goal tracking.

Wrong tool arguments. The model calls a tool with a plausible-sounding argument that is actually incorrect. This is worse in an agent context because tool errors compound. Validation (checking arguments before execution) and human approval for irreversible actions are not optional.

Looping. The agent re-performs the same action repeatedly, believing it has not yet succeeded, because it cannot correctly interpret the tool result. Explicit loop detection with a maximum action count is basic safety engineering.

Out-of-scope actions. The agent takes actions that were not intended by the user — sending messages, modifying files, making purchases — because the scope of the task was interpreted more broadly than intended.

Context window exhaustion. For very long tasks, the agent may run out of context window. Without explicit handling (summarization, retrieval), this causes the agent to “forget” earlier context and make decisions based on incomplete state.

Direct address: if your agent demo runs reliably in five steps, it will fail unpredictably at twenty. The debugging surface scales faster than the slide count. Invest in evaluation harnesses that test longer trajectories before deploying.

Evaluating agents

Single-turn LLM evaluation is already hard. Agent evaluation is harder for several reasons:

Non-determinism. Agents make choices at each step; different executions of the same task can take different paths. Evaluation must cover multiple trajectories, not a single fixed output.

Partial credit. An agent that completes 8 of 10 required steps correctly is partially successful. Binary success/failure metrics miss this.

Credit assignment. When an agent fails, which decision caused it? Tracing the root cause requires step-by-step inspection of the trajectory, not just outcome evaluation.

Environment fidelity. Agent evaluation requires a realistic environment for the agent to operate in. A test environment that is easier or harder than production will produce misleading results.

Effective agent evaluation uses: - Diverse starting states (not just the happy path). - Adversarial or edge-case scenarios. - Both outcome metrics (did the task complete?) and process metrics (were intermediate steps reasonable?). - Failure analysis to identify specific steps where agents consistently go wrong.

Open problems in agent research

- **Safe interruptibility:** Can an agent be stopped or redirected mid-task without causing harm from partially completed actions?
- **Long-horizon credit assignment:** How should feedback from a task outcome be attributed to individual decisions in a long trajectory?
- **Compositional generalization:** Can agents generalize from training tasks to novel combinations of skills they have not seen together?
- **Alignment under delayed reward:** If the agent's goal is rewarded only at task completion, how do we ensure intermediate actions remain aligned?

These are active research areas, not solved problems.

Takeaway

- Agents loop: observe, plan, act, update, decide. Each iteration involves a model that can be wrong.
 - Planning architectures (flat, CoT, ReAct, hierarchical) trade off simplicity and debuggability.
 - Memory architectures (in-context, episodic, semantic, working) add capability and complexity.
 - Primary failure modes: error compounding, goal drift, wrong arguments, looping, out-of-scope actions.
 - Agent evaluation requires multi-trajectory coverage, partial credit, credit assignment, and realistic environments.
 - Invest in evaluation harnesses before deploying agent systems at meaningful scale.
-

Try it

Exercise 1 — Multimodal evaluation design

You are building a vision-language feature for a retail product catalog: users upload photos of items and the model identifies the product and answers questions about it. Name two failure modes specific to this visual use case that would not appear in a text-only QA system. For each, describe how you would construct a test case to detect it.

Exercise 2 — Grounding protocol

Design a grounding evaluation for a retrieval-augmented system: a research assistant that answers questions by retrieving from a corpus of scientific papers. Describe the test cases you would include to verify that the model correctly abstains when the answer is not in the retrieved context, and what you would consider a passing result.

Exercise 3 — Agent failure trace

Describe a realistic failure scenario for a tool-using agent tasked with “schedule a meeting for next Tuesday with everyone on the project team.” Walk through the first five steps the agent might take, identify one specific failure mode, and describe one architectural change (not just a prompt instruction) that would make this agent more robust.

Exercise 4 — Memory architecture decision

A coding assistant agent needs to remember: (1) the user’s code style preferences from previous sessions, (2) the contents of files it edited in the current session, (3) documentation for the libraries it is using. For each of these three memory needs, identify the most appropriate memory type (in-context, episodic, semantic, working) and explain why. If two needs are best served by the same type, note the tradeoff.

End of Part IV. Previous: Part III — Efficiency: training, inference, and systems · Next: Part V — Frontiers and open problems · Or main volume.

Part V — Frontiers and open problems

Sharif Uddin

From Models to Frontiers · Volume III

The frontier is not only a line where capabilities end. It is also where evaluation fails to keep pace with capability, where scientific explanations lag behind empirical observations, and where honest researchers say “we do not know.” This final part covers evaluation at the frontier, the open problems that matter, and how to read a fast-moving field without either being swept away by hype or missing genuinely important progress.

Contents of this part

In the full volume table of contents, these correspond to sections 15–17.

	Chapter	What you will take away
1	Evaluation at the frontier	Capability vs. process evaluation; dynamic benchmarks; what leaderboards cannot tell you
2	Open research directions	Reasoning, continual learning, world models, human-AI collaboration
3	Reading the field and closing the trilogy	arXiv hygiene, sustainable curriculum, what the three volumes built

Contents (plain list — same as table):

1. Evaluation at the frontier — benchmarks, process evaluation, societal lens.
 2. Open research directions — reasoning, continual learning, world models.
 3. Reading the field — curriculum, sources, trilogy close.
-

Chapter 1 — Evaluation at the frontier

The benchmark leaderboard is a video game. High scores can coexist with bad outcomes, and bad scores can hide genuinely useful capabilities.

Evaluation is not a neutral measurement of capability. It is a claim that the thing being measured corresponds to something important. As model capabilities grow, the mismatch between what benchmarks measure and what matters tends to grow with them — and the stakes of getting this wrong increase.

Why standard benchmarks go wrong

Saturation. A benchmark that most models can nearly ace is no longer measuring meaningful differences. The variance it captures becomes noise rather than signal about capability gaps that matter for deployment. This happens faster than expected: what seemed like a challenging benchmark is often saturated within 18 months of its publication.

Contamination. Benchmark examples that appear in training data inflate performance artificially. Responsible evaluation uses held-out or dynamically generated benchmarks, but even held-out benchmarks from past papers eventually appear in web crawls as they accumulate citations and discussions.

Construct validity. A benchmark of math problems measures something. But “math problem performance” may not predict the capability that matters for your use case: the model may solve formatted textbook problems well while failing at the informal, multi-step, ambiguous mathematical reasoning in real scientific or engineering work.

Aggregation hides heterogeneity. An average score across 100 tasks tells you almost nothing about whether the model is uniformly good at 80/100 tasks or spectacular at 60 and terrible at 40. Two models with the same overall benchmark score can have very different profiles of which tasks they handle well.

Gaming and optimization pressure. When benchmarks become industry standards, they attract optimization. Some of this optimization produces genuine capability improvements — models get better at the thing the benchmark measures. Some of it produces benchmark-specific improvements that do not transfer.

Process evaluation vs. outcome evaluation

Outcome evaluation asks: did the model get the right answer? This is easy to automate and scores cleanly. The limitation: a model can get the right answer through wrong reasoning, and get the wrong answer through sound reasoning on a genuinely ambiguous question.

Process evaluation asks: was the reasoning process sound? This is harder to automate and does not always have a clean ground truth. The advantage: for tasks where correct reasoning matters (tutoring, medical advice, scientific analysis), a model that gets the right answer via reasoning errors is not a model you want to deploy — even if it passes outcome evaluation.

For high-stakes applications, process evaluation is often more relevant than outcome evaluation. A math tutor that gets the right answer by guessing is not a useful math tutor. A medical information system that reaches a correct recommendation through faulty reasoning cannot be trusted on cases where it is less likely to get lucky.

The practical challenge: process evaluation requires either human judgment or a second model that can evaluate reasoning quality — both of which are expensive and introduce their own biases.

Dynamic benchmarks

Dynamic benchmark construction attempts to create evaluation examples that have not appeared in training data and cannot easily be gamed:

- **Time-gating:** Evaluation examples from events after the training cutoff.
- **Procedural generation:** Generating new examples algorithmically from a specification, so the exact examples have not been seen.
- **Human adversarial data collection:** Having humans specifically try to find questions the model answers incorrectly, then using those as test cases.
- **Live competition benchmarks:** Benchmark examples from ongoing competitions that update continuously.

None of these fully solves the contamination and saturation problems, but they raise the difficulty of gaming and reduce the rate of artificial saturation.

The societal impact lens

Capability benchmarks measure what models can do. They do not measure what models do when deployed at scale to real users.

The gap matters because: - The distribution of real user queries is different from the distribution of benchmark questions. - The distribution of outcomes (who benefits, who is harmed, what behaviors are normalized) depends on the full deployment ecosystem, not model capability in isolation. - Social and economic impacts from AI systems are determined by deployment decisions, access patterns, and organizational choices, not by benchmark performance.

Occupational impact analysis, audit studies of AI-assisted decisions, longitudinal studies of usage patterns, and community feedback processes are research methodologies for evaluating at the deployment level rather than the model level. These are less developed and less standardized than capability benchmarking — partly because they are harder, and partly because they produce less convenient answers.

Friction: the leaderboard is a video game — you get points for winning the levels the game defines. Real deployment is a different game, on different levels, with different scoring. A model that dominates the leaderboard can still fail the people it serves.

Takeaway

- Standard benchmarks suffer from saturation, contamination, construct validity issues, and aggregation hiding capability heterogeneity.

- Process evaluation is more relevant than outcome evaluation for high-stakes applications where reasoning quality matters.
 - Dynamic benchmarks reduce (but do not eliminate) contamination and gaming.
 - Capability benchmarks measure potential in lab conditions, not impact under real deployment. Both matter; neither substitutes for the other.
-

Chapter 2 — Open research directions

Naming open problems honestly is a form of scientific integrity. The field generates confident claims faster than it generates validated answers.

This chapter names research directions where the problems are clear and important, the progress is real but incomplete, and bold claims should be interrogated. The goal is not pessimism — progress has been substantial — but calibration. Depth requires distinguishing what is understood from what is asserted.

Reasoning beyond pattern matching

The question: Do current language models reason, or do they pattern-match? Do they compute in a way that generalizes beyond training distribution, or do they find sophisticated statistical shortcuts?

What is understood: Models trained on chain-of-thought data produce longer, more structured reasoning traces that correlate with higher accuracy on many reasoning tasks. This is real and useful.

What is contested: Whether this constitutes genuine compositional reasoning — the ability to assemble known rules in new ways to solve problems structurally unlike anything in training — or whether it is sophisticated interpolation within the training distribution.

Why it matters: If models can generalize compositionally, their capabilities in novel domains may be substantially stronger than their training data directly supports. If they cannot, performance on distribution-shifted problems will be systematically worse than in-distribution evaluation suggests. This directly affects how much you can trust model performance on novel problems.

Current research directions: Systematic evaluation on problems designed to require out-of-distribution generalization (novel arithmetic structures, invented logical systems, causal reasoning with new relations), interpretability of reasoning traces, and architectures specifically designed for compositional generalization.

Continual learning

The question: Can a model update its knowledge and behavior in response to new information without degrading its performance on what it already knows?

The problem: Standard fine-tuning on new data causes **catastrophic forgetting** — the model overwrites existing knowledge in the process of learning new facts. The parameters that encoded old knowledge and the parameters that need to encode new knowledge overlap, and gradient descent on the new objective modifies both.

Current approaches: - **Parameter isolation:** Reserving specific parameters for new knowledge, either through architectural design or learned parameter masking. - **Rehearsal:** Including examples of old tasks in the fine-tuning data to prevent forgetting. - **Low-rank adaptation (LoRA):** Fine-tuning only a small number of parameters (low-rank weight perturbations), which limits interference with the base model's knowledge. - **Retrieval as continual learning:** Updating a retrieval index rather than the model weights, so new knowledge is non-parametric.

Why it matters: Deployed models need to stay current. The alternative — periodically retraining from scratch on accumulated data — is expensive and introduces instabilities. A model that can incorporate new information without forgetting old knowledge would substantially change the economics and operational complexity of AI deployment.

World models and physical reasoning

The question: Do language models have world models — internal representations of how the world works that support causal inference and planning, not just pattern association?

What is claimed: Models that can answer questions about physical systems, predict counterfactuals, and reason about cause and effect. These abilities exist and are useful.

What is contested: Whether the underlying representations support genuine causal reasoning (the ability to reason about what would happen under interventions — what if X had been different?) or whether models learn correlational patterns that approximate causal reasoning in in-distribution cases and break on novel causal structures.

Evidence from the field: Models succeed reliably at causal questions that have similar structures to training data. Performance degrades significantly on causal questions with novel structural features, or on tasks requiring counterfactual reasoning about physical processes that are unlike anything in text.

Why it matters: Planning, simulation, scientific reasoning, and many high-stakes applications require genuine causal inference. If model “understanding” of physical and causal structure is shallow, these applications will fail in predictable ways that benchmark performance does not surface.

Human-AI collaboration design

The question: How should humans and AI systems work together such that the collaboration is reliably better than either alone?

This is both a research question about human-computer interaction and an alignment question about how AI systems should behave in collaborative contexts. Key open problems:

Appropriate reliance: Users who over-rely on AI systems can miss errors that they would catch themselves. Users who under-rely fail to benefit from AI capabilities. Calibrating appropriate reliance requires both a well-calibrated AI (confident when right, uncertain when uncertain) and user training. Neither is fully solved.

AI-induced skill atrophy: If a professional consistently delegates a skill to an AI tool, does their own ability in that domain degrade? This is plausible for domains where practice maintains skill (writing, mathematical reasoning, judgment under uncertainty). The research is early and contested.

Collective knowledge effects: If many people delegate similar thinking to similar AI systems, does the diversity of human thought decrease over time? This is a macro-level question that cannot be studied in short-term individual experiments.

Disagreement and productive friction: For AI systems used in decision support, how should disagreement between the AI and the human be handled? A system that always defers to human judgment is not providing useful decision support; a system that pushes too hard on its recommendations undermines human agency.

Figure-caption aside: “here be dragons” is not cynicism — it is epistemic humility with a sense of humor. The dragons are the claims we do not yet know how to verify.

Generalization and emergence

Emergence — capabilities that appear at scale without being explicitly trained for — is real in the sense that performance on some benchmarks shows discontinuous jumps as model scale increases. It is contested in several ways:

- Some apparent emergence may be an artifact of evaluation methodology: using a metric that maps nonlinearly onto capability (e.g., exact-match accuracy on problems that require all steps to be correct) can produce apparent discontinuities even when underlying capability is smoothly scaling.
- Some emergent capabilities appear to emerge earlier in smaller models when evaluated with more sensitive metrics.
- The mechanistic explanation of why capabilities emerge at specific scale thresholds is not well understood.

The practical implication: be skeptical of claims that a specific capability will definitely emerge at a specific scale. Monitor empirically for capabilities as scale increases, and design systems to handle both their presence and absence gracefully.

Takeaway

- Compositional reasoning in novel domains is real but limited; current models may interpolate where we expect them to generalize.
 - Continual learning without catastrophic forgetting is an unsolved problem; retrieval is the current practical workaround.
 - World models and causal reasoning are present in shallow forms; deep causal reasoning on novel structures remains weak.
 - Human-AI collaboration introduces risks (over-reliance, skill atrophy) alongside benefits; these require deliberate design, not just capability.
 - Emergence is real but its mechanisms and predictions are contested; measure rather than extrapolate.
-

Chapter 3 — Reading the field and closing the trilogy

FOMO is a full-time job. Depth on the threads that matter to your work is how preprint consumption becomes judgment.

The field moves fast. A sustainable practice for following it requires selectivity, not comprehensiveness. This chapter gives you the reading strategy and then closes the trilogy.

Evaluating sources

arXiv is where most ML research appears first. The advantages: fast, open, comprehensive. The disadvantages: no peer review at the preprint stage, highly variable quality, and a volume that rewards click-optimized abstracts over careful content.

Signals for credibility at arXiv: - **Reproducibility:** Does the paper release code and evaluation scripts? Reproducibility is not sufficient for correctness, but an unreproducible paper cannot be verified. -

Related work engagement: Does the paper engage honestly with prior work and competing approaches, or does it cherry-pick comparisons? - **Ablations:** Does the paper show what components of the method are actually responsible for the improvement? Results without ablations are claims about a pipeline, not a method. - **Calibration of claims:** Does the paper distinguish between what the results show and what the authors believe? Overclaiming is common; a paper that acknowledges its limitations is more trustworthy.

Conference papers (NeurIPS, ICML, ACL, ICLR, CVPR) have been through peer review, which adds signal but is not infallible. The review process is compressed and relies on overworked reviewers; significant papers are occasionally rejected and weak papers accepted. Use conference acceptance as one signal, not the only one.

Blog posts and technical reports from labs are not peer-reviewed but are often the first published account of significant systems. Read them with awareness that they are marketing communications as well as technical writing. The most useful content is often in the methods and evaluation sections, not the headline results.

Model cards and system cards are the most directly actionable documents for practitioners. They describe what a model was trained on, how it was evaluated, what its known limitations are, and what uses it is and is not appropriate for. Read these before deploying any model in a production context.

Building a sustainable reading practice

The goal is not to read everything. The goal is to be well-informed on the threads that matter for your work while maintaining the time and mental bandwidth to think clearly about them.

A practical structure:

One conference track. Pick the venue and track most relevant to your work (alignment, systems, NLP, computer vision) and read the accepted papers list when it is published. You do not need to read every paper — the titles and abstracts will show you where to go deeper.

One or two trusted aggregators. Annotated paper newsletters (there are several in the ML space) or reading groups that summarize and filter can dramatically reduce the cost of staying current. The value is not just the papers they surface but the context they provide about what is significant.

One open-model release line. Follow the release series most relevant to your work (Llama, Mistral, Gemma, or others as they emerge). Read the technical reports, the model cards, and the evaluations. These provide grounded benchmarks for what is currently practical.

Primary sources for big claims. When a capability claim is cited enough to affect your decisions (a new reasoning approach, a safety result, a scaling breakthrough), read the original paper rather than the blog post summary. Summaries lose nuance; primary sources let you evaluate the methodology.

Direct address: if your list of sources is longer than you can actually read, cut it in half. What you cannot read cannot inform your judgment; it can only produce anxiety. Better to have three sources you read carefully than thirty you skim guilt-ridden.

What the three volumes built

This trilogy followed one thread from beginning to research depth.

Volume I — From Tokens to Understanding built the foundation: what tokens are, how attention works at an intuitive level, what the training process looks like, what models can and cannot do, and how to engage with them responsibly as an everyday user or early practitioner.

Volume II — From Prompts to Systems moved from foundation to practice: prompting as engineering discipline, retrieval and adaptation, evaluation, deployment, observability, cost, security, cross-functional teams, and responsible deployment. This is where most of the value is generated — by teams building carefully with the tools that exist.

Volume III — From Models to Frontiers stepped behind the API surface: the science of pretraining and scaling, the research foundations of alignment and safety, efficiency for training and serving at scale, multimodal models and agents, and the frontier's open questions. This is where the vocabulary for evaluating the field comes from.

The boundary between volumes is roughly: - Vol I → II: understanding what models are vs. building reliably with them. - Vol II → III: building with models vs. understanding or evaluating the models themselves.

You do not need all three volumes to be effective. Most of the value that LLMs generate in the world will come from teams working at the Vol II level — building carefully, measuring honestly, shipping responsibly. Volume III is for the readers whose work requires depth on the science, the safety research, or the research frontier.

A closing note on uncertainty

This volume named many open problems. That is intentional. The honest account of the field is not one of triumphant progress toward a fully understood technology. It is one of rapid capability growth, contested explanations, improving but imperfect evaluation, and genuine uncertainty about both the potential and the risks.

The researchers and practitioners doing the best work in this field hold two things simultaneously: substantial optimism about what these systems can do, and rigorous skepticism about whether any specific claim about those systems holds up to scrutiny. That combination — optimism paired with skepticism — is the hardest thing to maintain and the most valuable.

The bridge is open, in both directions.

Try it

Exercise 1 — Benchmark evaluation

Pick a benchmark commonly cited in model release comparisons (MMLU, HumanEval, GSM8K, HellaSwag, or another you have encountered). For that benchmark: identify one way it could be contaminated in a recent training crawl, one task type where it probably overestimates real-world capability, and one task type where it might underestimate. If you cannot find all three, that is informative — note which you could not identify and why.

Exercise 2 — Process evaluation design

A company uses a large language model to help analysts write investment research reports. The current evaluation measures whether the model's outputs contain specific required sections and passes a factual accuracy check against company databases. Design a process evaluation that would supplement this — what would you evaluate about the reasoning process itself, not just the final output?

Exercise 3 — Open problem impact

Choose one open research problem from Chapter 2 (compositional reasoning, continual learning, world models, human-AI collaboration, emergence). Describe one specific product or application that currently exists where closing that research gap would change what is practically possible. Be specific about what currently fails and what would become feasible.

Exercise 4 — Reading list audit

List the sources you currently use to stay current on LLM research. For each source, categorize it: primary (papers, model cards), secondary (summaries, newsletters), or marketing-adjacent (lab blogs, press releases). If more than half are in the marketing-adjacent category, identify one primary or secondary source to add and one marketing-adjacent source to deprioritize. If your list is empty, start with one model card and one conference accepted papers list.

End of Part V — Volume III — From Models to Frontiers. Previous: Part IV — Beyond text: multimodal models and agents · Trilogy hub — README · Or main volume.

Notes

Key terms used throughout this volume

Scaling law — An empirical relationship linking compute, data volume, model size, and loss. Useful for budgeting training experiments and reasoning about tradeoffs. Not a promise about any specific downstream task or domain.

Chinchilla-optimal — A training allocation that roughly balances model parameters and training tokens for a given compute budget. Derived from research showing that many earlier large models were trained on too little data relative to their size.

Pretraining — Training a model on massive amounts of text with a next-token prediction (or similar) objective. Produces a base model that encodes broad statistical patterns from language; not directly deployable without post-training.

Instruction tuning (SFT) — Supervised fine-tuning on demonstrations of desired behavior. Teaches the model what task format looks like, how to respond to instructions, and basic dialogue conventions.

RLHF / DPO — Preference-based post-training methods. RLHF (reinforcement learning from human feedback) trains a reward model on human comparisons then uses it to update the policy. DPO (direct preference optimization) achieves a similar goal without a separate reward model. Both aim to steer outputs toward human-preferred behavior.

Alignment — The broad project of making models behave in accordance with intended goals — helpful, honest, harmless in common shorthand. Not guaranteed by scale; requires deliberate design and ongoing evaluation.

Emergence — Capabilities that appear at scale without being explicitly trained for, as measured by a discontinuity on some benchmark. The definition and mechanism are actively debated.

Hallucination — Model outputs that are confidently stated and factually wrong. A symptom of the model generating plausible-sounding text without a ground-truth check, not a bug that will be fully “fixed.”

KV cache — The key-value pairs from the attention mechanism stored in memory during inference, so the model does not re-compute previous context on each new token. A major component of inference memory cost.

Quantization — Reducing the numerical precision of model weights (e.g., from 32-bit floats to 8-bit integers) to reduce memory and speed up computation. Involves a quality tradeoff that varies by task and precision level.

Red teaming — Structured attempts to elicit harmful, policy-violating, or otherwise undesired behavior from a model. Both a safety practice and an ongoing research methodology.

Model card — A document accompanying a model release that describes its intended use, training data, evaluation methodology, limitations, and known risks. A minimum standard of transparency for foundation model releases.

How to use this volume

Read it as a map, not a textbook. Each chapter explains a landscape — what the key techniques are, what they trade off, what the open questions are — not how to implement every detail. Primary sources (papers, model cards, documentation) remain the authoritative reference.

Read primary sources alongside it. When a chapter references Chinchilla scaling results or a specific alignment technique, that is a pointer to read the original work. The chapter gives you framing; the paper gives you detail.

Return to Volumes I and II. Frontier topics connect back to shipping constraints regularly. When a Part III efficiency technique connects back to a Vol II deployment decision, that is the connection the book is trying to build.

Track the field selectively. Part V covers reading strategy in detail. The short version: one trusted aggregator, one conference track relevant to your work, one open-model release line you follow. Depth beats breadth.

Start with Part I — Scale, data, and the pretraining stack. Or start with the part that covers your gap.
Previous volume: From Prompts to Systems — Volume II.